



# 实验一、Mininet应用实践

主讲人:黄辉

email:huanghui@wzu.edu.cn

# 实验环境

- 虚拟机软件（vmware workstation）；
- 虚拟机镜像（SDNHub\_tutorial\_VM\_64-bit[NEW](可直接导入VMware).ova）
- 镜像下载链接：

[https://pan.baidu.com/s/1qYN\\_MtUboPmruHda1DgrTA](https://pan.baidu.com/s/1qYN_MtUboPmruHda1DgrTA)

提取码：mhfi

# 实验目的

- 能够运用mininet可视化工具创建网络拓扑结构
- 能够运用mininet交互界面创建网络拓扑结构
- 能够运用python脚本构建网络拓扑结构

# 实验内容

- **基础技能：**

- 用可视化工具miniedit创建一个网络拓扑；
- 命令行创建拓扑；
- 交互式界面创建主机、交换机等
- 对节点间进行ping测试

- **进阶技能：**

- 编写python脚本完成对于计算机网络拓扑结构的构建

- 详见实验指导书。

# 什么是Mininet?

- 软件定义网络（SDN）的仿真工具。
- 创建一个真实的虚拟网络，其中包含真实的工作组件，但运行在一台机器上，以便于测试。
- 提供通过以下方式创建主机、交换机和控制器的能力：
  - 命令行 如： `sudo mn`
  - 交互式用户界面 如： `py net.addHost('h3')`
  - python应用程序 如： `sudo python lab_1.py`
- 更多信息： <http://mininet.org/>

# 测试Mininet是否正常工作

- 要快速检查它是否工作，请键入以下命令：
  - `sudo mn --test pingall --topo single,3`
- 此命令将：
  - 创建一个单一网络，其中3台主机以非交互模式连接到单个交换机。
  - 执行从所有主机到所有其他主机的ping。
  - 此命令使用默认的交换机与控制器。

# 执行过程

```
ubuntu@sdnhubvm:~[07:33]$ sudo mn --test pingall --topo single,3
```

```
*** Adding controller
```

```
*** Creating network
```

```
*** Adding hosts:
```

```
h2 h3 h4
```

```
*** Adding switches:
```

```
S1
```

```
*** Adding links:
```

```
(s1, h2) (s1, h3) (s1, h4)
```

```
*** Configuring hosts
```

```
h2 h3 h4
```

```
*** Starting controller
```

```
*** Starting 1 switches
```

```
S1
```

```
*** Ping: testing ping reachability
```

```
h2 -> h3 h4
```

```
h3 -> h2 h4
```

```
h4 -> h2 h3
```

```
*** Results: 0% dropped (0/6 lost)
```

```
*** Stopping 3 hosts
```

创建三（3）个主机和一个交换机。

定义节点之间的链接。

启动默认控制器。

执行从所有节点到所有节点的ping操作。

# 基本命令

- **help**—显示可能的mininet命令列表。
- **nodes**—显示节点列表。
- **net**—显示网络拓扑（ASCII编码）。
- **dump**—显示每个节点的接口设置和表示每个节点的进程的PID。
- **<node id> ifconfig**—类似于用于定义网络接口的Linux命令。
- **<node id> route**—类似于Linux命令，用于定义节点路由表中的条目。
- **iperf <node id> <node id>**—两（2）个节点之间的网络性能测试。
- **<node id> ping <node id>**—两（2）个节点之间执行ping测试。
- **pingall**—在网络中的所有主机之间执行ping测试。
- **xterm <node id>**—启动登录到该指定节点的远程xterm。

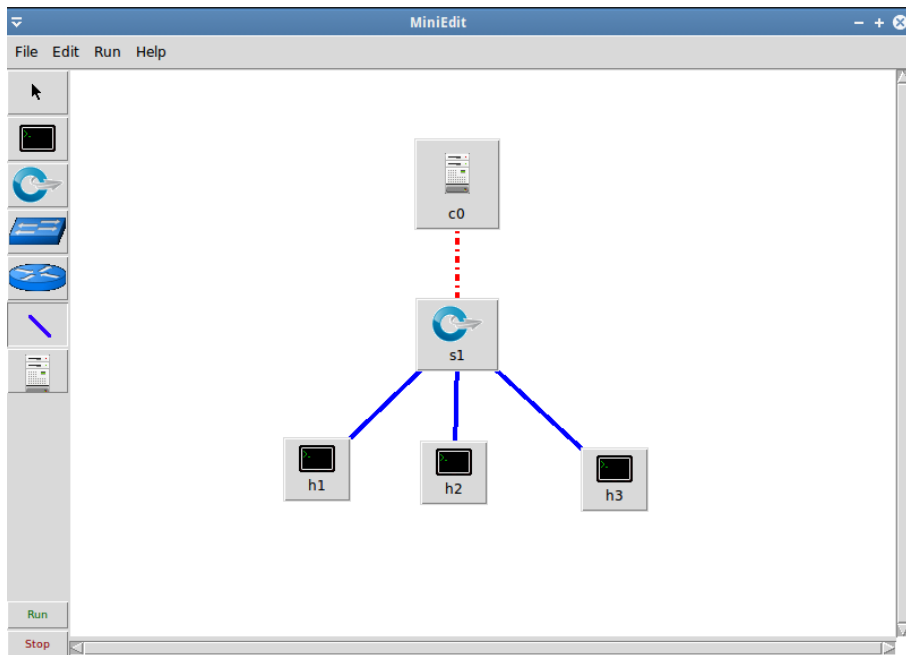
# 基本命令行参数

- **--topo**—在mininet启动时通过命令行定义拓扑。
- **--switch**—定义要使用的交换机。默认情况下，使用ovsk软件交换机。
- **--controller**—定义要使用的控制器。如果未指定默认控制器，作为集线器使用。

# 可视化工具创建拓扑

- `cd mininet/examples/`
- `sudo python miniedit.py`

“File”->“Export Level 2 Script”导出python文件

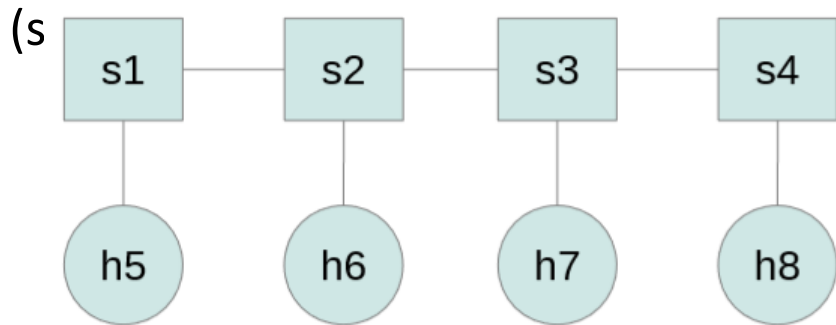


# 命令行创建拓扑

- 最小网络，两（2）台主机连接到一（1）台交换机。
  - `sudo mn --topo minimal`
- 每个主机都连接到一个交换机，所有交换机都彼此连接。
  - 在本例中，有4个主机和4个交换机。
  - `sudo mn --topo linear,4`
- 每个主机都连接到一个交换机。
  - 在本例中，有3个主机和1个交换机。
  - `sudo mn --topo single,3`
- 具有定义深度和扇出的基于树的拓扑。
  - `sudo mn --topo tree,depth=2,fanout=2`

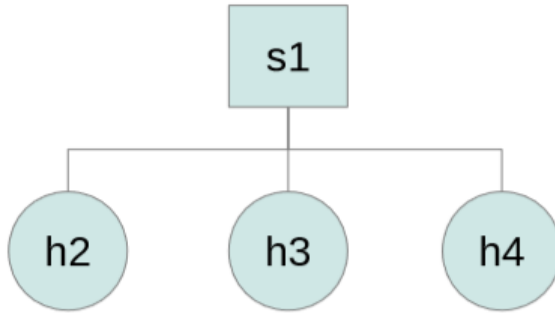
--topo linear,4

(s1, s2) (s1, h5) (s2, s3) (s2, h6) (s3, s4)



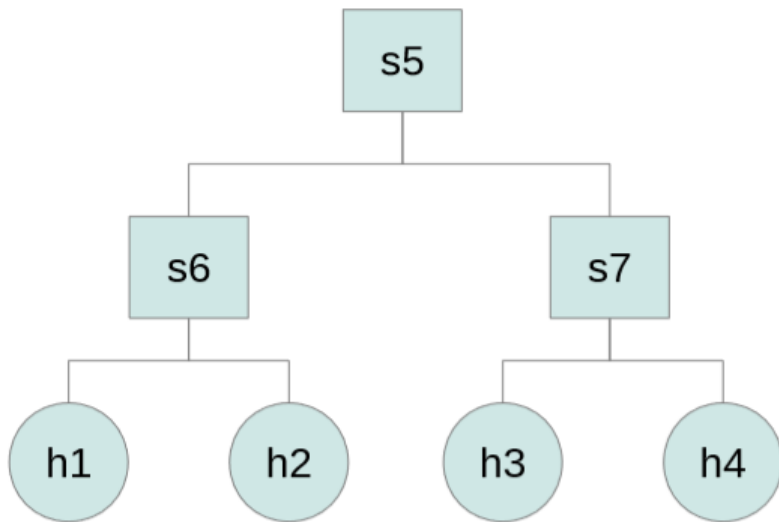
--topo single,3

(s1, h2) (s1, h3) (s1, h4)



--topo tree,depth=2,fanout=2

(h1, s6) (h2, s6) (h3, s7) (h4, s7) (s5, s6)



# 交互式创建拓扑

```
$ sudo mn -v output
```

```
mininet> py net.addHost('h3')
```

添加主机h3

```
<Host h3: pid=3405>
```

```
mininet> py net.addLink(s1, net.get('h3'))
```

添加s1和h3之间的链接

```
<mininet.link.Link object at 0x1737090>
```

```
mininet> py s1.attach('s1-eth3')
```

```
mininet> py net.get('h3').cmd('ifconfig h3-eth0 10.0.0.3')
```

h3配置IP地址

```
mininet> h1 ping -c1 10.0.0.3
```

h1 ping h3

```
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
```

```
64 bytes from 10.0.0.3: icmp_req=1 ttl=64 time=1.91 ms
```

# 脚本创建任意拓扑

```
from mininet.net import Mininet

net = Mininet()

# Creating nodes in the network.

c0 = net.addController()

h0 = net.addHost('h0')

s0 = net.addSwitch('s0')

h1 = net.addHost('h1')

# Creating links between nodes in network

net.addLink(h0, s0)

net.addLink(h1, s0)

# Configuration of IP addresses in interfaces

h0.setIP('192.168.1.1', 24)

h1.setIP('192.168.1.2', 24)
```

- 能够创建所需的任何拓扑。
- 请注意添加链接的两（2）种方法。
- 默认情况下，脚本会在完成后删除网络。可以将`mininet.cli.cli（net）`放在`net.stop（）`之前，以切换到交互模式。
- 这个网络是什么样子的？

# 设置网络性能限制

```
from mininet.net import Mininet
from mininet.node import CPULimitedHost
from mininet.link import TCLink

net = Mininet(host=CPULimitedHost, link=TCLink)

c0 = net.addController()
s0 = net.addSwitch('s0')
h0 = net.addHost('h0')

h1 = net.addHost('h1', cpu=0.5)
h2 = net.addHost('h2', cpu=0.5)

net.addLink(s0, h0, bw=10, delay='5ms', max_queue_size=1000, loss=10, use_htb=True)
net.addLink(s0, h1)
net.addLink(s0, h2)

net.start()

net.pingAll()

net.stop()
```

- 注意，`addhost()` 语法允许您指定：
  - 每个虚拟主机的CPU（CPU）分配百分比
- 请注意，`addlink()` 语法允许您指定：
  - 带宽（`bw`），单位为Mbps
  - 延迟（`delay`）
  - 最大队列大小（`max_queue_size`）
  - 损失（`loss`）百分比

# 网络测试

# display network information

```
mininet.util.dumpNodeConnections(net.hosts)
```

# testing ping from all hosts to all hosts

```
net.pingall()
```

# test ping from one host to another host

```
net.ping(h0, h1)
```

# test packet performance from one host to another host

```
net.iperf((h0,h1))
```

- 对programmatically机制获得的信息与mininet网络进行测试
  - mininet.util.dumpnodeconnections ( ) 用于以可打印的方式返回所有网络信息。
  - mininet.net.mininet.pingall ( ) 是从所有节点到所有节点的ping测试，而mininet.net.mininet.ping ( ) 是从一个主机到另一个主机的ping测试。
  - mininet.net.mininet.iperf ( ) 是一个基本的包测试。

# 其他

- `net = Mininet(controller=RemoteController, switch=OVSKernelSwitch)`
  - 能够自定义控制器或交换机
  - `mininet.node.RemoteController(ip=?, port=?)`提供使用外部OpenFlow控制器（如POX）的机制。
  - `mininet.node.UserSwitch`提供使用用户空间交换机而不是默认内核交换机的机制。
  - `mininet.node.OVSSwitch`提供一种机制，用于使用预装有MiniNet的OpenVswitch。
- `mininet.cli.CLI(<Mininet object>)`
  - 允许脚本进入交互模式
  - 通常放在<Mininet object>.stop()前面

谢谢！